

Frontier Coding Agents Don't Have a Retrieval Problem: The Localization Ceiling and Where Cost Actually Lives

Yash Doke
Independent Researcher
yashdoke215@gmail.com

August 2026

Abstract

Structural code retrieval—handing a coding agent the exact functions it needs instead of letting it read its way to them—is a natural way to cut what the agent spends, and an active line of research and product work pursues it. SKELETONGRAPH runs lexical, dense-semantic, and call-graph retrieval simultaneously, returns whole functions, and serves them over the Model Context Protocol. We measure what that buys inside a real agent: Claude Code in headless deployment, 100 SWE-bench Verified tasks, every candidate fix adjudicated by executing the project's own test suite.

Retrieval quality improves sharply. The first search returns the correct *file* for 86% of tasks against 66% for the agent's built-in text search, and names the correct *function* in roughly 80% against 0%—text search matches lines and returns files, so function-level localization is not something it does poorly but something it cannot do. Cost does not follow. The median task is no cheaper (+1.9%); the entire saving sits in the expensive tail (−42.5% at the 95th percentile), and solve rate is a statistical tie.

Because SKELETONGRAPH runs every non-LLM retrieval paradigm at once, we can ask whether the remaining limit is our index or the category. Stripping the symbol names from the issue text, and repeating the comparison on a decontaminated benchmark of unseen repositories, drops its first-search recall from 86% to 55%, with text search falling alongside it; issuing more searches does not recover the gap. Memorization and location cues turn out to be one quantity—knowledge of where to look, held by the model or supplied by the issue—and where neither is present the agent acquires it by exploring instead. A retrieval layer supplies candidate files, not knowledge of the repository, and the cost saving persists throughout, so what it reduces is how far the agent wanders rather than how well it finds code. We release the harness, task lists, and per-task records.

1 Introduction

An AI coding agent solving a real repository issue spends its budget in three places: finding the code to change, writing the change, and running tests until they pass. Tools that improve the first of these are now a small industry, and they advertise large savings—commonly an order of magnitude, sometimes over 99%. Almost all of those figures come from the same calculation: count the tokens a structured query returns, count the tokens it would take to read the corresponding files end to end, and report the ratio.

That calculation leaves out the agent, and the omission matters more than it appears. An agent does not read whole files by default, so the denominator describes a workflow nobody runs. It does not stop once it has found the code, so the numerator covers a fraction of the trajectory. And it

is not billed per retrieved token: it is billed for the entire conversation, re-sent on every step. A retrieval improvement is therefore a claim about one phase of a three-phase process, reported in units nobody is charged in.

This paper asks what better retrieval is actually worth once the agent is put back in. We compare retrieval strategies inside a single harness that holds everything else fixed—the same model, the same repository tasks, the same edit-and-submit interface—and we do not accept a system’s own judgment of whether it solved a task: every candidate patch is applied and the project’s real test suite is executed in a container.

Throughout, we compare against what we call the agent’s *built-in tools*: the text search, file glob, and file read that a production coding agent ships with and is heavily trained to use. This is a deliberately strong baseline. Much of the published work in this area compares a retrieval system against reading entire files, which no competent agent does; a retrieval layer that only beats an artificially weakened baseline has not been evaluated.

What we find. Structural retrieval works. On its first search, built-in text search recovers 66.3% of the files a task requires and can never name the function to change—text search matches lines and returns files, so function-level retrieval is not something it does badly, it is something it cannot do. Our system recovers 86.2% of the files on its first search and identifies the correct function in roughly 80% of tasks. Against seven alternative strategies it consumes the fewest tokens by a wide margin.

And yet the money moves much less than that suggests, and not where one would expect. Deployed inside a production agent over 100 tasks, retrieval makes simple tasks slightly *more* expensive and removes roughly 40% of the cost of the hardest ones. Break-even falls around \$0.35 per task—the 66th percentile of baseline task cost here, and a trajectory of about twelve turns: below it the agent was already locating the code quickly and retrieval is overhead, above it the agent was hunting and retrieval pays, increasingly so. Solve rate does not change at all.

The explanation turns out to be simple, and is visible in a single measurement: the peak context the agent accumulates is the same with retrieval as without it (44.1k versus 44.9k tokens). Retrieval does not shrink what the agent carries; it shortens the path to the answer. Cost falls in proportion to the steps saved and no further. Underneath that is a fact about how agents consume context: an agent acquires whatever the task requires, from us if we supply it and by itself if we do not, and anything acquired early is re-sent on every step that follows. Give it less and it fetches the difference itself, adding steps; give it more and the surplus is charged repeatedly. Until the agent’s own step count falls, the bill keeps growing regardless of retrieval quality.

Contributions.

1. **A decoupling result.** We show that large gains in retrieval accuracy do not produce proportional cost reduction in a deployed agent, and we quantify the gap: unchanged median cost against a 42.5% reduction at the 95th percentile (Section 5.2).
2. **An explanation, not just an observation.** We give a cost model for agentic retrieval, show that the context an agent carries is bounded below by what the task requires, and derive why payload shaping has limited leverage in both directions while step count does not (Sections 4.4 and 4.5).
3. **Evidence that the benchmark cannot see retrieval.** A control that receives no repository code at all solves 36–44% of tasks on the standard benchmark. Retrieval quality is not

measurable through solve rate here—which also invalidates solve-rate comparisons that would favor us (Section 5.3).

4. **A localization ceiling, and evidence it is categorical.** Removing the code symbols from an issue degrades our three-paradigm retriever sharply and significantly—0.125 within task, 95% CI [0.010, 0.260]—and additional searches do not recover it. Running lexical, dense and structural retrieval simultaneously therefore does not solve symptom-only localization, which locates the limit at similarity-based localization rather than at our implementation and identifies the open problem as reasoning over a representation of program behavior—not a better index (Section 6.4). Crossing this factor with whether the model has memorized the repository yields the sharpest form of the decoupling: across all four conditions, including a decontaminated benchmark and the conditions where our retrieval margin is a few points of recall, the cost saving persists at -21% to -31% . The unifying account is that both factors supply the same thing—knowledge of where to look—and that a retrieval layer supplies candidate files rather than that knowledge. Where it is absent the baseline acquires it by exploring, and does so well enough that its cumulative recall overtakes ours.
5. **Five negative results with diagnostic value.** We attempt to convert retrieval quality into savings five ways and report all five failures, including one where removing a genuinely wasteful step made outcomes worse because the waste was load-bearing (Section 6.3).
6. **A reproducible artifact.** The harness, the frozen task lists, and per-task records are released,¹ along with an index that any user can rebuild from source because building it never calls a language model.

On the title, and the word “frontier” in it. The title is a claim about a specific regime, and the qualifier is load-bearing. In the frontier-agent deployment the model has substantially memorized these repositories, so retrieval quality cannot express itself in outcomes: solve rate is a tie, and the median task is no cheaper despite first-search file recall rising from 66% to 86% and function-level localization from 0% to roughly 80%. That is the sense in which retrieval is not the binding constraint there.

It is emphatically *not* a claim that retrieval never matters, and this paper contains the counter-example. On the open-weight model of Section 5.4, which memorizes far less, a control receiving no repository code solves 35.0% of tasks and structural retrieval reaches 42.0%—a genuine seven-point effect (Table 5). Retrieval is worth little precisely where the model already knows the code and a great deal where it does not. Readers who take only one sentence from the title should take that one. What we establish is stated precisely in Section 6.4, with the observation that would falsify it, and the limits of its generality in Section 7.

2 Background: What a Code-Context System Is For

2.1 The task and where its cost comes from

A repository-scale coding agent is given an issue and repository access, and must produce a patch that passes hidden tests. Its trajectory has three phases that consume budget very differently:

1. **Localization** — find the code to change. Cost scales with how much of the repository the agent must inspect before it is confident.

¹Code, task lists, and per-task run records: <https://github.com/yashdoke7/skeletongraph>.

2. **Implementation** — write the edit. Cost scales with how much code must be understood well enough to modify correctly.
3. **Validation** — run tests, read failures, repair. Cost scales with how many repair iterations are needed, and is unbounded in the worst case.

A retrieval layer can only act directly on phase 1. This is obvious once stated, but the prevailing evaluation practice obscures it: reporting retrieval token savings in isolation implicitly assumes phase 1 dominates. A central goal of this paper is to measure how much of the budget phase 1 actually is.

2.2 The design space, and what each design optimizes

Existing systems are not arbitrary—each embodies a hypothesis about which term in the cost dominates. Making those hypotheses explicit predicts each system’s cost profile before any measurement.

Prompt injection (repository maps). Aider [1] builds a PageRank-ranked skeleton of the repository and injects it into the prompt. *Hypothesis:* the agent’s problem is not knowing what exists, so give it a map up front. *Implied cost:* the map is part of the fixed prompt and is therefore re-sent every turn — a one-time modelling decision billed T times. Our measurements confirm this is the most expensive strategy tested by a wide margin.

Graph query (knowledge-graph servers). Codebase-Memory [2] and Graphify [3] extract an entity graph and expose queries over it. *Hypothesis:* the agent’s problem is navigating relationships, so answer relationship queries on demand. *Implied cost:* pay-per-query rather than up-front, but query results are entity-shaped rather than edit-shaped, so payloads tend to be broad. The two differ in what building the graph costs, and the distinction matters enough to state precisely rather than lump together. Codebase-Memory builds its graph with tree-sitter and adds semantic edges with a code embedding model—no generative model is invoked, which puts its index construction in the same category as ours. Graphify extracts with a language model, chunk by chunk, once per repository; on a large repository that is a substantial one-time cost, and it is excluded from the token savings such systems report. We attribute it to that arm explicitly.

Lexical search (grep, BM25). *Hypothesis:* identifiers are highly discriminative in code, so text search is sufficient. This is a stronger baseline than it appears; Xia et al. [4] show that a simple hierarchical localize-repair pipeline is competitive with elaborate agentic scaffolding, [?] find text search generally outperforming vector retrieval inside several production agent CLIs, and our own results show lexical navigation reaching the gold file in the large majority of tasks. *Implied cost:* cheap per call, but file-granular — the agent must read to narrow further, and those reads are the dominant token sink we measure.

Structural retrieval (this work). *Hypothesis:* the unit an agent edits is a *function*, so retrieval should return functions, ranked by combined lexical, semantic and structural evidence. This is supported independently: Townsend et al. [5] find function-level localization yields the highest repair rate against both line-level and file-level granularity, while noting the optimum is task-dependent; and Chen et al. [6] show graph-guided localization reaching high file-level accuracy at reduced cost. *Implied cost:* pay-per-query with edit-shaped payloads. Our contribution is to measure what that buys end-to-end, and to report honestly that it buys less than the design predicts.

2.3 What the literature already suggests about the ceiling

Two prior findings anticipate our result without stating it in cost terms. Wang et al. [7] evaluate retrieval-augmented code generation broadly and report that although high-quality context improves generation, retrievers struggle to fetch useful context *and generators fail to exploit additional context effectively*. Townsend et al. [5] similarly find diminishing and task-dependent returns as localization granularity sharpens. Both point to the same place: the bottleneck migrates out of retrieval. Our contribution is to measure that migration in dollars and turns inside a real agent loop, and to explain it with a cost model.

3 Related Work

Agentic software engineering. SWE-agent [8] introduced agent-computer interfaces for repository tasks; OpenHands [9] generalized the platform. Xia et al. [4] argued the opposite direction—that much agentic machinery is unnecessary—with a three-phase localize/repair/validate pipeline competitive with agent scaffolds at low cost. We inherit that scepticism: our baseline is a strong native-tools agent, not a weakened one, because a retrieval layer that only beats a hobbled baseline has not been evaluated.

Localization as a first-class problem. LocAgent [6] parses repositories into heterogeneous graphs for multi-hop localization, reaching up to 92.7% file-level accuracy with a fine-tuned 32B model at roughly 86% cost reduction relative to proprietary models. Townsend et al. [5] isolate granularity itself and find function-level best on average but task-dependent. Our function-level results (Figure 2) are consistent with both. Where we differ is the dependent variable: these works measure localization accuracy, while we measure what accuracy converts into once an agent must also implement and validate.

Retrieval-augmented code generation. CodeRAG-Bench [7] is the closest methodological precedent—a holistic benchmark separating retrieval quality from end-to-end generation. Their finding that generators under-exploit retrieved context is, in our framing, the generation-side counterpart of the cost-side ceiling we measure.

Code-context products. Codebase-Memory [2], Graphify [3], and Aider’s repository map [1] are the deployed systems in this category. They differ from this work in evaluation rather than ambition: headline numbers are token-count comparisons against whole-file reading, self-graded, or measured without an agentic loop. Where an agentic evaluation is reported, a quality cost accompanies the saving—one such report describes roughly 10× fewer tokens at 90% of baseline answer quality. Our result differs in kind: a smaller cost reduction with *no* measurable quality loss, and paired per-task records that make the comparison checkable.

Memory and context management. MemGPT [10] treats context as a paging problem. SKELETONGRAPH is deliberately narrower—a structural index and reranker, not a memory manager—and zero-LLM at index time, so index construction is never billed to a model. One result from that literature is worth stating because it rhymes with ours in a different setting: HierMem [11] reports that for long-horizon conversational agents, *which* material is placed and held in context governs instruction compliance more than how much material is retrieved. Our finding has the same shape on the cost axis rather than the quality axis—retrieving better does not reduce spend,

while bounding what the agent does before it commits does—which suggests the naive lever in both settings is retrieval volume and the effective one is control over what the trajectory carries.

Benchmarks and contamination. SWE-bench [12] is the standard benchmark; its contamination is now widely discussed and major evaluators have moved away from reporting SWE-bench Verified as a headline. Decontaminated successors such as SWE-rebench [13] draw continuously from thousands of repositories with tasks postdating model cutoffs. Our no-retrieval control quantifies the contamination effect directly in our own setting (Section 5.3).

4 Method

4.1 Design principle: a systems comparison, not a tool ablation

Each pipeline keeps its native retrieval interface. Baselines (`grep`, `bm25`, `hybrid`) receive the harness’s standard tool set; competitor systems receive their own tools (graph query, snippet fetch, repo-map injection); SKELETONGRAPH receives its own search and expand tools. What is held constant is the substrate: model, task set, edit/submit interface, turn budget, and verification. This measures *deployed systems*, the unit practitioners actually choose between, rather than a retrieval function in isolation.

4.2 SkeletonGraph

SKELETONGRAPH has three stages, shown in Figure 1: an offline index build, a per-query retrieval path, and a serving layer.

Index build (offline, zero-LLM). Tree-sitter [14] parsers—covering ten language families—turn each source file into a symbol table of fully-qualified names with line ranges, signatures, and docstrings, together with a call graph over which we compute PageRank centrality. A BM25 [15] index and, optionally, a code-specialized dense embedding (jina-code) are built over the same symbols; embeddings are content-hash cached so only changed functions re-encode. The whole index is persisted and, because no language model is invoked, is rebuildable by any user from source. This is a substantive difference from the LLM-extracting graph system in Section 2.2, whose per-repository extraction pass is excluded from its reported token savings and must be re-paid on material change; it is *not* a difference from every graph system, since Codebase-Memory also builds its index without a generative model.

Retrieval (per query). An issue query is run through three legs in parallel—BM25 (lexical), the dense encoder (semantic), and a structural reranker that reorders a BM25 recall pool by entity anchoring, PageRank, and call-graph proximity—and their ranked lists are combined by reciprocal rank fusion [16]. Fusion is chosen over a tuned linear combination deliberately: RRF has no score-scale parameters to fit, so no part of the ranking is tuned on the evaluation set. We evaluate two operating points: *SG-Fusion* uses all three legs, and *SG-Rerank* drops the dense leg (a BM25 pool reordered by structural confirmation), requiring no embedding model or prewarm.

Serving. The retrieval surface is exposed to agents over the Model Context Protocol [17], which is what makes the deployment measurement in Section 5.2 possible against an agent we do not control.

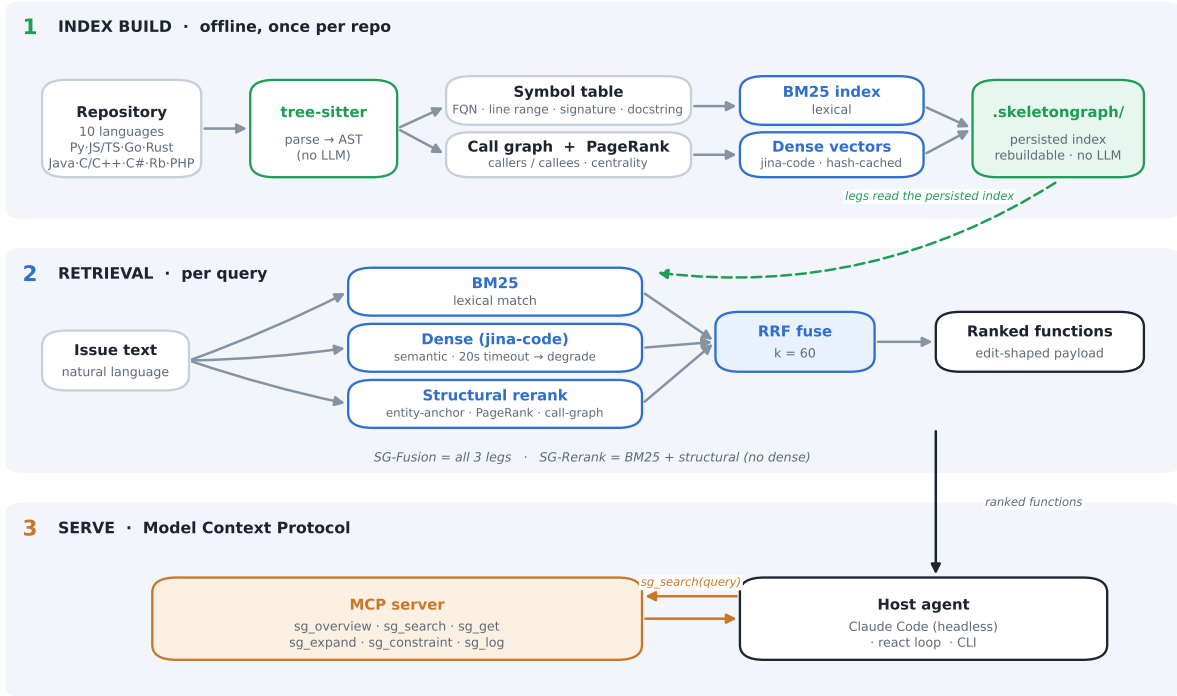


Figure 1: SKELETONGRAPH architecture. The index build (top) invokes no language model; the retrieval path (middle) fuses three signals over the persisted index; the serving layer (bottom) exposes the result to a host agent over MCP.

4.3 Harnesses

React loop. Our own agent loop offers tools directly to the model, with no MCP handshake. This is the axis on which competitor systems can be compared fairly (Section 5.5 explains why they cannot be compared inside a headless frontier agent).

Frontier-agent deployment. SKELETONGRAPH runs as a real MCP server driven by a production frontier coding agent—Claude Code in headless (-p) mode, invoked as `--model sonnet`, which the run transcripts record as resolving to `claude-sonnet-5` for every task reported here—against the same agent, same model, using only its unmodified built-in tools. This measures deployment economics under an agent we do not control; subsequent references to “the frontier agent” denote this configuration.

What “built-in tools” means, and where each number comes from. Two baselines recur and must not be conflated. In the deployment study the baseline—which tables label *built-in tools* and the released run records tag **native**—is Claude Code driving its own shipped file-navigation tools: its text search (**Grep**), file glob, and file read, with no MCP server attached. It is not a reimplement and not a weakened stand-in; it is the product as shipped, and it is a strong baseline precisely because the model is heavily trained on those tools. In the react loop the corresponding baselines are separate arms we implement—**grep** and **bm25**—running on `nvidia/nemotron-3-super-120b-a12b`, an open-weight mixture-of-experts model with 120B total and 12B active parameters. Every retrieval, cost, turn and context figure in this paper belongs to exactly one of these two settings; we never compute a comparison across them, and each table

states which one it reports.

The two harnesses implement the same retrieval algorithms in separate code paths and their numbers are never pooled. Unless a table states otherwise, SKELETONGRAPH is deployed as *SG-Fusion*—the frontier-agent deployment measurement in Section 5.2 is SG-Fusion; Table 2 separately reports SG-Rerank on the agent-free intrinsic retrieval ablation.

4.4 A cost model for agentic retrieval

Before measuring, it is worth writing down what an agent is actually billed for, because the discrepancy between that quantity and the quantity competitors report is the origin of the whole result.

Let a trajectory consist of turns $t = 1 \dots T$. At each turn the model is sent the entire accumulated conversation. Write c_t for the context size at turn t . Billed input tokens are then

$$N = \sum_{t=1}^T c_t = \sum_{t=1}^T \left(c_0 + \sum_{s < t} r_s + \sum_{s < t} a_s \right), \quad (1)$$

where c_0 is the fixed prompt overhead, r_s the tokens returned by the tool called at turn s , and a_s the model’s own output at turn s .

It matters what c_0 contains, because one component of it is charged to the retrieval layer. c_0 is the system prompt *together with the serialized definitions of every tool the agent can call*—and registering a retrieval server adds its tool schemas to that fixed prefix, where Equation (1) bills them once per turn for the whole trajectory. A retrieval layer therefore starts each task in deficit, before it returns a single result. Our measurements include this: token counts are the entire prompt the model receives, so SKELETONGRAPH’s seven tool definitions are charged to SKELETONGRAPH, and the comparison understates it rather than flattering it. Two further consequences follow immediately, and both are borne out empirically.

Tool output is charged once per remaining turn, not once. A result r_s appears in every subsequent context, so its true cost is $r_s (T - s + 1)$. Retrieved tokens are therefore *not* interchangeable with saved read-tokens: an early, large payload is the most expensive object in a trajectory. We use this weighting for attribution in Section 6.

Reducing r alone cannot reduce N much. Competitor claims optimize $\sum_s r_s$: the token cost of retrieval versus reading whole files. But in Equation (1) that term is one of three, and it is not the dominant one once c_0 and the accumulated a_s are included. Writing $\bar{c}$ for mean context size, $N \approx T\bar{c}$, so

$$\frac{\Delta N}{N} \approx \frac{\Delta T}{T} + \frac{\Delta \bar{c}}{\bar{c}}. \quad (2)$$

A retrieval layer can act on either term: it can shorten the trajectory (fewer turns to localize) or slim the context (less material carried). Our central empirical finding, Section 5.2, is that structural retrieval moves the first term and leaves the second at approximately zero—which bounds the achievable saving at the turn reduction, independent of how good retrieval gets. This is a structural ceiling, not a tuning failure, and it predicts the observed -21% turns / -24% tokens correspondence before we measure it.

Why the median and the mean must be reported separately. Equation (1) is quadratic in T when a trajectory wanders: extra turns both add their own context and re-send everything prior. Task cost is therefore heavy-tailed, and a mean over tasks is dominated by its upper tail. A retrieval layer that truncates long trajectories will move the mean substantially while leaving the median untouched. Reporting only a mean—as the aggregate percentages in this literature do—cannot distinguish “everything got cheaper” from “the disasters stopped,” which are very different products.

4.5 The irreducible context floor

Equation (2) leaves open how far $\bar{c}$ can be pushed down. We argue it is bounded below, and that the bound is set by the model’s task rather than by the retrieval system’s cleverness.

An agent must acquire enough material to write a correct edit. Call that quantity c^* : the minimum context sufficient for the task. A retrieval layer chooses what to hand over, but it does not choose c^* — that is a property of the bug and of what the model needs to see to be right. Two regimes follow:

- If the layer returns *less* than c^* , the agent does not fail quietly: it goes and gets the difference itself, with its own tools. The tokens reappear as reads and greps, and the extra retrieval turns are added on top. Cost is relocated, not saved.
- If it returns *more* than c^* , the surplus is not merely wasted once. By Equation (1) it is re-sent on every remaining turn, and the surplus is delivered *early* — so it carries the largest possible multiplier.

This is the sharpest practical consequence of the model, and it explains a result that surprised us. A payload delivered at turn 1 is billed roughly T times. An agent behaves exactly as designed: it consumes what it is given, and everything it consumes early persists in context for the rest of the trajectory. Consequently, **shaping the payload has bounded leverage in both directions** — below c^* the agent recovers the shortfall itself, and above c^* the surplus is amortized against the agent rather than against the retrieval layer. The only term with unbounded leverage is T .

And T is not a retrieval variable. After localization, turn count is governed by implementation and validation: how many edits the model attempts, whether it runs tests, how it reacts to a failure. A retrieval layer influences T only through the localization prefix. Once that prefix is short, further retrieval improvement acts on a term that has already gone to zero.

Section 6.3 reports five attempts to beat this bound — two that push the payload above c^* , two that push it below, and one that declines to retrieve at all on the tasks where retrieval does not pay — and the failure mode of each is the one this section predicts.

4.6 Metrics and statistics

We report Docker-verified pass@1; per-task cost, turns, and total input tokens; peak context; file- and function-level retrieval hit rate; first-search precision; and per-tool call counts. Token counts are the full prompt the model receives each turn—system prompt, tool definitions, accumulated conversation and all tool results—not retrieval payloads in isolation. Cost is the runtime’s own reported billing in the deployment study and is imputed from token counts under a fixed published price sheet in the react loop; the two are never pooled, and every reported change is paired within a single harness (Section 7 gives the full accounting).

Following Section 4.4 we report the full cost distribution (p50/p75/p90/p95), not only means. Paired comparisons use McNemar’s exact test for pass@1 and a paired bootstrap (10^4 resamples of

	Built-in tools	+ SKELETONGRAPH
Correct file found by the <i>first</i> search	66.3%	86.2%
Correct file found <i>eventually</i> (any search)	83.4%	90.7%
Correct <i>function</i> identified	0%	79–81%
Precision of the first search	61.8%	34.2%

Table 1: Retrieval accuracy inside the production agent, 100 tasks. File rows are *recall*, not “at least one file matched”.

task *pairs*) for cost and turn deltas. We resample pairs and recompute the aggregate ratio rather than averaging per-task percentage changes, because the mean of per-task ratios diverges from the ratio of means when some tasks are near-zero cost—a difference large enough in our data to flip the sign of the reported effect.

5 Results

5.1 Retrieval quality: what each approach can and cannot find

The first question is whether structural retrieval finds the code better than the text search an agent already has. It does, and Figure 2 shows the result has two distinct parts that are easy to conflate.

Three points about how Table 1 is measured, before reading it. File rows report *recall*—the fraction of a task’s gold files returned, averaged over tasks—rather than the weaker criterion of at least one file matching, which flatters any retriever that returns many files. The SKELETONGRAPH first-search figure excludes three tasks in which the agent never invoked SKELETONGRAPH at all; those are adoption events rather than retrieval failures, and scoring them as misses would attribute a decision the retriever did not make to its accuracy. The function row is a range because gold function names recovered from patch hunks are approximate: 81% against the released gold set, 79% after a stricter disambiguation pass. Both are reported; neither changes a conclusion.

At the level of *which file to edit*, both approaches work, and the gap depends on when you measure. On its first search SKELETONGRAPH finds the right file 86.2% of the time against 66.3% for text search—but given the whole trajectory, the baseline eventually reaches it 83.4% of the time. That difference between 66.3% and 83.4% is precisely what the baseline’s hunting costs: both usually reach the right file, but one returns it and the other finds it by reading. Text search is a genuinely strong baseline, consistent with prior work finding simple pipelines competitive with elaborate scaffolding [4]. The difference is not whether it arrives but how much it spends arriving, and Section 6 shows those intervening reads are the single largest token consumer in the baseline.

At the level of *where in the file to edit*, the gap is not one of degree. SKELETONGRAPH names the correct function in about 80% of tasks; text search names it in zero. This is not a tuning deficit: text search matches lines and reports the files containing them, so a function is not an object it can return. That function-level granularity is the right target is supported independently—Townsend et al. [5] find function-level localization produces higher repair rates than either file-level or line-level.

This accuracy is not free, and the price is paid in precision, which runs the other way: 34.2% for SKELETONGRAPH against 61.8% for built-in search. Read plainly, SKELETONGRAPH much more reliably includes the right code and also includes more code that turns out not to be needed. It finds the right neighbourhood and hands over the neighbourhood. Section 6.2 shows this is exactly the mechanism behind its worst individual cases—the trade a structural retriever makes is breadth

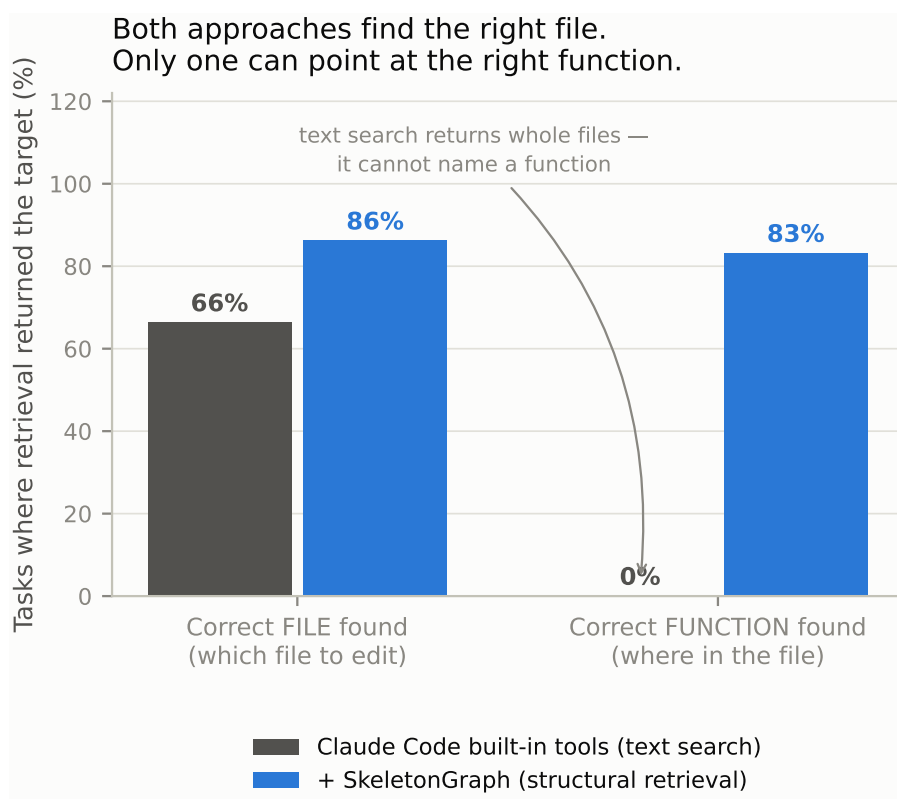


Figure 2: Retrieval accuracy at two granularities (frontier agent, 100 tasks). Lexical search scores 0% at function level by construction, not by tuning.

in exchange for reliably covering the target.

Why lower precision is not simply waste, and what recall actually buys. It would be natural to read that precision figure as pure overhead, and on the cheapest tasks it is. But identifying the file to edit is not the same as knowing what to change in it. To write a correct patch the model has to work out which behaviour is wrong and why, which generally means reading the code around the target—its callers, the helper it delegates to, the test that exercises it. An agent given only the responsible file still has to assemble that surrounding material before it will act, and it does so by reading. Handing over the neighbourhood supplies some of it up front. This is why the precision gap does not translate into a proportional cost penalty: part of what a precision metric scores as unnecessary is material the agent would otherwise have gone and fetched.

It also clarifies what higher recall is worth. Section 6 shows the agent’s peak context is essentially identical with and without retrieval, so retrieval is not making the working set smaller. What it changes is *when* the working set is assembled: the agent reaches the point of having enough to act in fewer steps, and because every step re-sends everything accumulated so far, fewer steps is where the money is. Better recall front-loads context rather than reducing it—and on a task whose localization prefix was already short, front-loading buys nothing while the extra tool definitions and retrieval calls still cost, which is the +1.9% at the median.

The same ordering holds with the agent removed entirely. Table 1 measures retrieval as the deployed agent actually used it—one call among many, mixed with whatever the agent did

Retrieval backend	MRR	Recall@10
Lexical grep	0.159	0.348
BM25	0.482	0.719
SG-Rerank	0.518	0.824
BM25 + Dense	0.551	0.843
BM25 + Dense + SG (fusion)	0.658	0.856

Table 2: Intrinsic retrieval ablation, no agent: same 100 SWE-bench Verified tasks as Table 1, backend varied with everything else fixed.

Task cost percentile	Cost per task (USD)		Change
	Built-in tools	+ SKELETONGRAPH	
50th (the median task)	\$0.255	\$0.260	+1.9%
75th	\$0.581	\$0.489	−15.8%
90th	\$1.010	\$0.752	−25.6%
95th (the worst tasks)	\$1.559	\$0.896	−42.5%
Average across all tasks	\$0.434	\$0.371	−14.6%

Table 3: Where the saving lives: the cost distribution over the same 100 tasks. Paired bootstrap 95% CI on the average change: $[-25.3\%, -1.2\%]$.

around it. To check that the ranking is a property of the retrievers and not an artifact of how one particular agent happened to call them, we run a controlled, agent-free ablation over the same 100 tasks: five backends, identical queries, identical k , no agent in the loop at all.

The ordering is exactly what Section 2.2’s design-space argument predicts, and it is monotone: each signal added to the ranking improves both metrics, and combining all three beats any subset. This rules out the alternative explanation for Table 1—that the frontier agent’s own search habits, rather than the retriever, produced the gap—and it is the cleanest single piece of evidence that structural and semantic signals are additive with lexical search rather than redundant with it, at least up to the ceiling Section 6.4 identifies.

5.2 The decoupling: retrieval improves, the typical bill does not

Given that retrieval accuracy improves as much as it does, the natural expectation is that cost falls correspondingly. It does not, and the shape of what happens instead is the paper’s central result.

The headline average is a 14.6% cost reduction across 100 paired tasks. That average is misleading on its own, in a specific and instructive way: SKELETONGRAPH is cheaper on only 40 of the 100 tasks, and the cost of the median task rises very slightly (+1.9%). Nearly the entire average saving comes from a small number of expensive tasks. Table 3 shows the distribution.

The effect is monotone in task difficulty, and it crosses zero. Table 4 groups the same 100 tasks into deciles by what the baseline spent. The trend is close to perfectly ordered, and it shows something the summary statistics hide: on easy tasks SKELETONGRAPH is not merely neutral, it is *consistently more expensive*.

Two things follow. First, there is a **break-even complexity** at roughly \$0.35 per task: below it retrieval is overhead, above it retrieval pays, and the return grows with difficulty. Second, the

Decile	Baseline cost range	Cost change	Tasks where SKELETONGRAPH was cheaper
1	\$0.10 – \$0.12	+44.0%	0 / 10
2	\$0.12 – \$0.14	+55.1%	0 / 10
3	\$0.14 – \$0.16	+22.0%	0 / 10
4	\$0.16 – \$0.18	+18.0%	2 / 10
5	\$0.19 – \$0.25	+95.3%	2 / 10
6	\$0.26 – \$0.31	+18.7%	4 / 10
7	\$0.31 – \$0.41	−10.5%	7 / 10
8	\$0.43 – \$0.67	−25.9%	8 / 10
9	\$0.68 – \$0.96	−28.0%	8 / 10
10	\$1.01 – \$2.12	− 42.8%	9 / 10

Table 4: The crossover: tasks grouped into deciles by baseline cost. Break-even falls near \$0.35 per task.

ordering of the win-count column (0,0,0,2,2,4,7,8,8,9) is what makes this a law rather than an artifact—a noisy effect would not order itself that cleanly across ten independent groups. One caveat belongs with the table: decile 5 contains a single volatile task that inflates its percentage, though its win count remains consistent with its neighbours—which is why the ordered win counts, not the percentages, carry the argument.

The mechanism is straightforward once stated. SKELETONGRAPH adds a fixed overhead: its own retrieval calls, which are billed whether or not they were needed. On a \$0.11 task that overhead is a large fraction of the total. On a \$2.00 task it is negligible, and what dominates instead is whether the agent found the code without a prolonged hunt. Restricting to tasks where the baseline spent at least \$1.00, the saving is 42.8%; above \$1.50, 45.0%. Turn count falls 21.4% overall (95% CI [−30.0%, −10.5%]) and total tokens 23.6%.

Figure 3 makes the shape visible by ordering tasks from cheapest to most expensive: the two cost curves lie on top of each other for roughly the first two thirds of tasks and separate only at the end. Figure 3 shows the same 100 tasks individually against a line of equal cost.

Why the expensive tasks are where the saving is. The mechanism is not that retrieval works better on hard tasks. It is that expensive tasks are expensive *because the agent could not find the code*, and that is the one failure retrieval prevents. When the baseline localizes quickly, its trajectory is short and there is nothing to save; SKELETONGRAPH adds its own retrieval calls and comes out marginally behind, which is the +1.9% at the median. When the baseline cannot localize, it searches, reads, searches again, and each of those steps re-sends everything accumulated so far—the quadratic behaviour of Equation (1). Those are the runs that reach \$1.50 and beyond, and they are exactly the runs where handing the agent the right function immediately collapses the trajectory. A concrete instance appears in Section 6.2: a task on which the baseline spent 50 turns and \$1.84 exploring, and SKELETONGRAPH finished in 14 turns and \$0.43.

A practitioner reading only the −14.6% average would reasonably expect their bill to fall by about a seventh across the board. The correct expectation is that most tasks cost the same and the occasional runaway task stops running away.

The measurement that explains the ceiling. Section 4.4 decomposed cost into how many steps the agent takes and how much context it carries per step. We can now say which of the two retrieval actually moves, and the answer is unambiguous. The peak context the agent accumulates

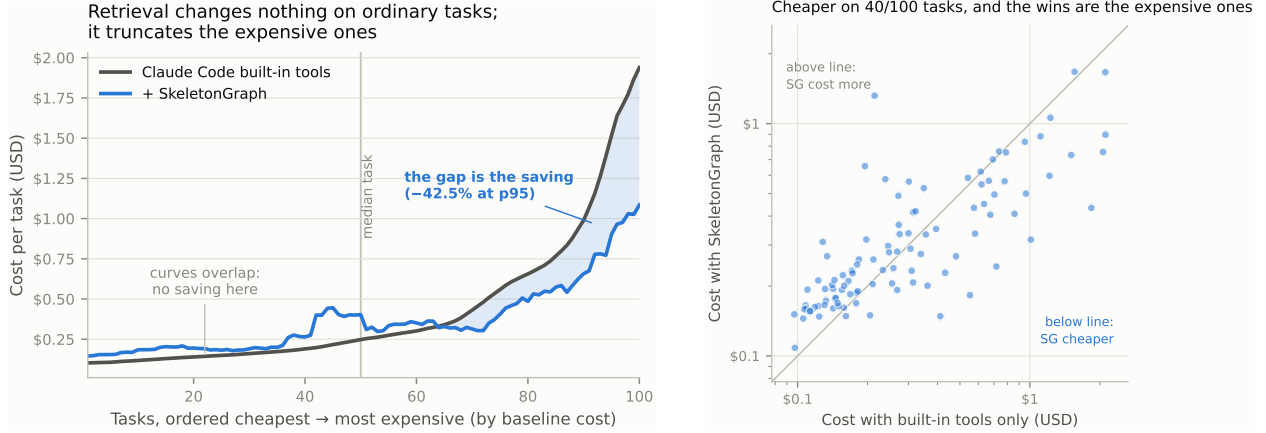


Figure 3: Left: cost by percentile. The shaded region is the saving; it opens only above the median. **Right:** the same 100 tasks, paired. Points below the line are tasks where SKELETONGRAPH was cheaper. Both panels are the same finding at different resolutions.

is 44,129 tokens with built-in tools and 44,892 with SKELETONGRAPH: the same, to within 2%.

This is worth stating in plain terms, because it is the paper’s explanatory centre. *Retrieval does not give the agent a smaller working set.* The agent ends up holding just as much material either way. What changes is how quickly it assembles that material—21.4% fewer steps—and cost falls by almost exactly that proportion, 23.6%. The two numbers matching is not a coincidence; it is the identity in Equation (2) with one of its two terms measured at zero.

The consequence is a ceiling that no amount of retrieval quality can lift. If the context term does not move, then cost reduction cannot exceed step reduction, and step reduction is bounded by how much of the trajectory was spent searching in the first place. Once localization is fast, the remaining steps belong to writing the fix and running the tests, and a retrieval layer has no purchase on either.

5.3 Solve rate is a tie, and the benchmark explains why

Solve rate is statistically indistinguishable: 75/100 for SKELETONGRAPH versus 74/100 native, with 68 shared solves, 7 SKELETONGRAPH-only and 6 native-only (McNemar exact, $p = 1.0$). We do not claim a solve-rate improvement.

The reason is benchmark saturation. In the react loop, a `none` control receiving *no repository code at all* solves 36–44% of tasks, against 42–45% for the best retrieval arm. Retrieval is worth a few points at most here, because the tasks are drawn from twelve heavily-studied open-source repositories the model has substantially memorized. Any solve-rate comparison between retrieval systems on SWE-bench Verified is therefore measuring memorization plus noise, not retrieval.

5.4 A systems comparison on a second model family

The deployment result compares SKELETONGRAPH against the agent’s built-in tools. To compare against *other retrieval systems* we use the react loop, where every pipeline is offered its own tools directly and no MCP handshake can exclude a competitor (Section 5.5). Figure 4 places each system on the two axes that matter jointly.

Three things in this figure are worth stating explicitly.

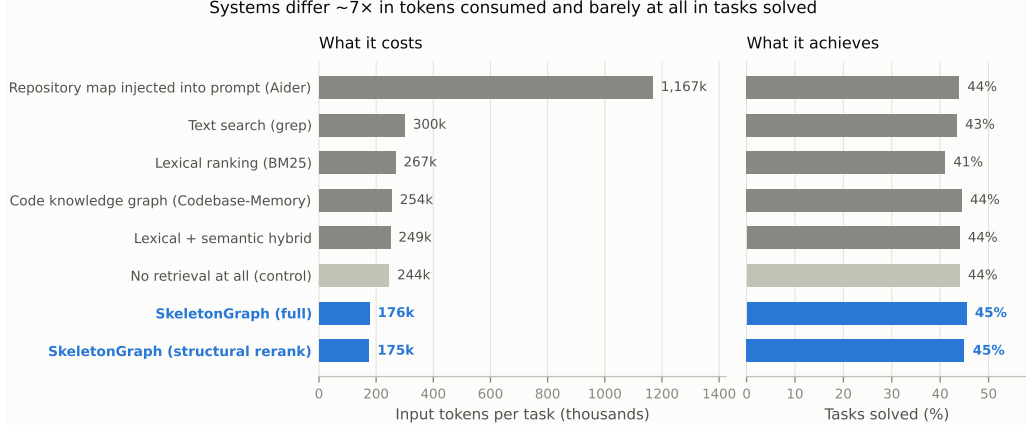


Figure 4: Whole systems on a second model family (**nemotron-3-super-120b-a12b**, 100 tasks; each pipeline keeps its own retrieval interface).

Retrieval backend	n	pass@1	rec@1	funcHit	Cost
SG-Fusion	100	42.0%	0.737	57%	\$0.052
BM25	100	41.0%	0.642	43%	\$0.074
Graph-query competitor	100	41.0%	0.223	9%	\$0.078
Lexical grep	100	39.0%	0.647	0%	\$0.079
Repo-map competitor	98	36.7%	—	—	\$0.160
none (closed-book)	100	35.0%	0.000	0%	\$0.066

Table 5: Controlled retrieval ablation in the react loop: **nemotron-3-super-120b-a12b**, 100 tasks, identical action space, only the retrieval backend varies.

The vertical axis is nearly flat, including for the control. Solve rates span roughly 41–45% across every system tested, and the **none** arm—which receives no repository code at all—sits at 44%, within noise of the best retrieval pipeline. On this benchmark, retrieval is worth about one point of solve rate. We regard this as the single most important methodological fact in the area, and return to it below.

The horizontal axis is not flat at all. Systems differ by roughly $7\times$ in tokens consumed to reach the same outcome. Aider’s repository map is the clearest case: injecting a PageRank-ranked skeleton into the prompt costs $\sim 1.17\text{M}$ input tokens per task against SKELETONGRAPH’s $\sim 176\text{k}$. Equation (1) explains why—an injected map is part of c_0 and is therefore re-sent on *every* turn, so a one-time “context” decision is billed T times. Graph-query systems (Codebase-Memory, $\sim 254\text{k}$) sit between the two: they fetch on demand rather than injecting, but return broader payloads than a function-level fetch.

Retrieval quality and token cost are separately optimizable. SG-Fusion sits at the best-recall, lowest-cost corner of this figure simultaneously—the agent-free intrinsic ablation (Table 2) confirms the same ordering with no agent search habits in the loop. Retrieval quality is real and measurable; it simply is not what determines whether the task is solved on this benchmark.

Table 5 reports the controlled ablation. Unlike the frontier-agent setting, this model memorizes less: the closed-book floor is 35.0%, and retrieval buys a genuine seven-point improvement on top

of it. SG-Fusion is simultaneously the highest-scoring arm, the cheapest arm, and the only one that localizes to the function rather than the file. This is the cleanest evidence in our study that retrieval quality can matter for outcomes—and it matters precisely where memorization does not already supply the answer.

5.5 Why competitors cannot be compared inside a headless frontier agent

Two graph/LSP-based systems we wired as MCP servers never participated: the headless agent finalizes its tool manifest within roughly two seconds of launch, and servers with slow initialization (language-server startup, Node CLI spawn) are absent from that manifest. They recorded zero tool calls—structural exclusion, not non-adoption. We report this as a deployment finding rather than a quality comparison, and it is why competitor quality is measured in the react loop instead.

The general lesson—that the harness can matter as much as the retriever—has independent support. [?] compare text and vector retrieval across a custom loop and three production agent CLIs and find that accuracy depends strongly on which harness and tool-calling style is used even when the underlying data are held fixed. Our exclusion result is an extreme instance of that effect: a harness property decided whether a retriever participated at all. It is also why we never pool numbers across our two harnesses, and why we treat the deployment magnitudes in Section 5.2 as specific to the agent we measured rather than as properties of retrieval in general.

6 Analysis

6.1 Cost tracks turns, and context is invariant

Peak context is effectively identical with and without retrieval (44,129 vs 44,892 tokens). Retrieval does not produce a leaner context; it produces a shorter trajectory. Since total input tokens accumulate as approximately turns $\times$ context, and measured turn reduction (21.4%) closely matches measured token reduction (23.6%), turn reduction is a hard ceiling on achievable cost reduction. This single observation explains why a 0% $\rightarrow$ 80% improvement in function-level retrieval yields only a 15% cost change.

Tool-level attribution supports the same conclusion. Weighting each tool result by the number of subsequent turns it is re-sent across—the $r_s(T - s + 1)$ weighting of Section 4.4—native cost is dominated by file reading (72.9% of carried tool tokens), which SKELETONGRAPH displaces (Figure 5). But SKELETONGRAPH’s own responses then account for roughly 70% of its carried tool tokens: the expenditure is relocated, not eliminated. This is the mechanism behind $\Delta\bar{c} \approx 0$. A retrieval layer substitutes its payload for the reads it prevents, and to first order the substitution is even.

6.2 When breadth is misread as scope

Section 5.2 reported that SKELETONGRAPH is more expensive on 60 of 100 tasks, mostly by small margins. Those cases have a consistent cause, and it follows from the precision measurement: retrieval returns the right region, and the agent occasionally infers that the whole region needs changing.

The clearest instance is a Django task about form handling. SKELETONGRAPH retrieved, at rank one, the base class method responsible for altering database schema—an entirely correct result, and an architectural hub called by every database backend. The agent took the hub’s centrality as evidence that the fix belonged at that level, and edited across the base implementation plus the

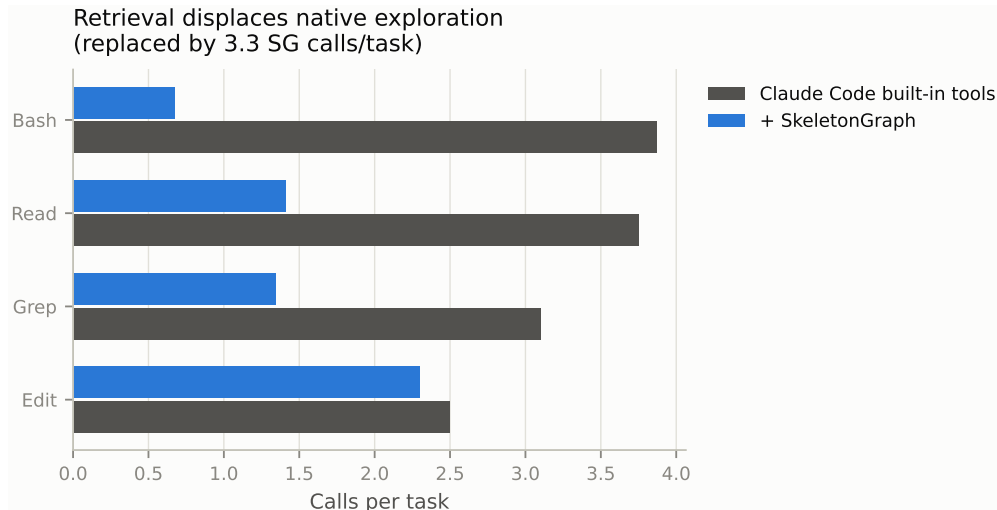


Figure 5: Where the turns go. Structural retrieval collapses exploratory navigation—shell, read, and grep calls all fall sharply—and replaces it with a small number of targeted retrieval calls. Editing is unchanged, which is consistent with the residual cost living in implementation rather than search.

MySQL, Oracle, PostgreSQL, and PostGIS backends. It spent 38 turns and \$1.32. The baseline, which found a narrower entry point by text search, made a smaller change in 12 turns and \$0.21. Retrieval was not wrong; it was broad, and breadth was read as scope.

The same property produces the wins. On an Astropy coordinate-transformation task, the baseline spent 50 turns and \$1.84 searching for the relevant modules; SKELETONGRAPH returned them immediately and the task finished in 14 turns and \$0.43. Handing over a region is decisive when the agent would otherwise not find it, and counterproductive when the agent would have found something tighter on its own.

This asymmetry—modest losses on tasks that were already easy to localize, large wins on tasks that were not—is the tail profile of Table 3 restated at the level of individual runs, and it is why the average and the median disagree.

6.3 Five attempts to beat the bound, and why each failed

Section 4.5 predicts that payload shaping has bounded leverage and that T is not a retrieval variable. We tested that prediction directly, with five interventions spanning the space. All five are reported; none succeeded. Their failure modes are individually informative and jointly diagnostic.

(A) Give more, and forbid the alternative. We inlined function bodies into search results and disallowed native text search, forcing retrieval to be the sole path to code. *Prediction:* pushes the payload above c^* and removes the agent’s recovery route. *Outcome:* cost fell 19% and solve rate fell with it, by two tasks—the same two in each of two independent runs, so not a sampling artifact. Removing the recovery route converts a retrieval miss from a recoverable detour into an unrecoverable failure, and the inlined bodies were billed on every subsequent turn.

(B) Remove the need to re-read. Returned bodies originally lacked per-line line numbers, so an agent that wanted to construct an exact edit re-read the file to obtain them. We measured this: 81% of all re-reads were of material already returned. Adding line numbers removed the need. *Prediction:* a clean win — same information, fewer tokens. *Outcome:* cost fell, and solve rate

fell with it — five losses against zero gains across 44 paired tasks. Transcripts explain why: the re-read was not only fetching line numbers, it was the step during which the agent paused, looked at surrounding code, and then ran the failing test. Removing the mechanical need removed the incidental behavior. The agent edited and submitted without executing anything.

This is the most instructive of the four. The "waste" we identified was real, and eliminating it was still harmful, because the wasteful action was load-bearing for an unrelated and more valuable behavior. Efficiency interventions on agent trajectories cannot be validated on the metric they target.

(C) Restore verification explicitly. If the verification loop was what (B) destroyed, mandate it: instruct the agent to reproduce the failure, edit, and re-run before submitting. *Prediction:* recovers (B)'s losses while keeping its savings. *Outcome:* substantially worse — input tokens rose 90% with no solves recovered. The instruction converted a calibrated behavior into an unconditional one: the agent ran test sweeps on tasks that did not need them, and the phrase "confirm nothing else broke" induced defensive edits far beyond the bug. Verification was already happening at roughly the right rate; making it mandatory removed the model's judgment about *when*.

(D) Give only pointers. The converse of (A): return only file, symbol and line range — never bodies — and let the agent read what it decides it needs with its own tools. *Prediction:* pushes below c^* ; the agent recovers the difference, and tokens relocate rather than disappear. *Outcome:* as predicted. On narrow, well-scoped tasks the leaner payload was competitive, but on broad tasks the agent made a small number of retrieval calls, then reverted to extended native navigation — and total cost rose. Withholding bodies does not save the tokens; it moves them from a targeted retrieval call into a longer sequence of untargeted reads, while adding turns. We report this one as a trajectory-level observation rather than a measured delta: unlike (A)–(C) it was not run as a full paired sweep in the deployment harness, and we do not attach a percentage to it.

(E) Route each task to whichever system suits it. The four above shape *what* retrieval returns. A fifth option is to decide *whether* to retrieve at all: given the crossover in Table 4, send cheap tasks to the built-in tools and expensive ones to SKELETONGRAPH. This is the obvious product response, so we measured its ceiling before building it.

An oracle allowed to see both outcomes and pick the cheaper per task spends \$31.80 against \$37.07 for always using SKELETONGRAPH—a 14.2% gain, and that is an upper bound no implementable rule can exceed. It comes with a defect that rules it out on its own terms: the oracle's solve rate is 72/100, *below* both always- SKELETONGRAPH (75) and never-SKELETONGRAPH (74). Cheap runs are frequently cheap because the agent gave up early, so selecting for low cost selects for failure.

Implementable rules do far worse. No feature available before the run separates the cases: whether the issue names the target file differs by 5 percentage points between SKELETONGRAPH's wins and losses (70% versus 65%), issue length by 4%. Signals from SKELETONGRAPH's own first search are no better—it locates the gold file in 95% of its wins and 88% of its losses. Every threshold rule we simulated on these signals was worse than simply always retrieving.

The reason is structural, and it is the same wall from a new direction: SKELETONGRAPH's retrieval is *equally good* on cheap and expensive tasks. Expensive tasks are not ones where retrieval fails; they are ones where the change is larger—1.54 gold files on average against 1.10, and 23.3 baseline steps against 5.7. Difficulty lives in implementation and validation, so no retrieval-time signal can forecast it. Predicting cost from retrieval state is a category error.

What the five together show. The interventions move in every available direction. (A) and (C) add; (B) and (D) subtract; (E) declines to act at all on some tasks. Every one degraded cost or correctness, and the one with a favourable ceiling reaches it by selecting failures. That is the signature of an optimum, not of insufficient tuning.

Taken with the measurements, this is the evidence for a genuine ceiling rather than an unexploited opportunity. Four independent observations point at it:

1. Retrieval accuracy is already near its practical maximum—86.2% first-search file recall, 80% function identification—and the median task is no cheaper. Whatever remains is not waiting on better ranking.
2. The agent’s peak context is unchanged (44.1k versus 44.9k tokens), so the term that would need to move in Equation (2) does not move.
3. SKELETONGRAPH’s own responses become roughly 70% of the tokens it carries forward. It substitutes for the reads it prevents rather than eliminating them.
4. Both directions of payload shaping, and task routing on top, fail.

We state the implication plainly, because it is easy to lose in a results table: **beyond a certain point, cost stops responding to retrieval quality at all.** An agent takes what the task requires regardless of who supplies it, holds it for the rest of the run, and spends the remainder of its budget implementing and validating. A retrieval layer buys the localization prefix and nothing after it—and on tasks where that prefix was already short, it is a net cost.

6.4 The ceiling belongs to the paradigm, not to our implementation

A natural objection is that our structural retriever is simply not good enough, and that a better lexical index, a stronger code embedder, or a richer graph would close the gap. We can address this directly, because SG-Fusion is not one paradigm but three operating simultaneously: lexical (BM25), semantic (a code-specialized dense encoder), and topological (the call graph). If any of the three could infer which code is responsible for a described symptom, removing the symbol names from the issue text should not much matter.

We test this by crossing two factors: whether the model has memorized the repository (the standard benchmark versus a decontaminated one), and whether the issue text still contains location cues (the original text versus a prose-stripped variant with tracebacks and code blocks removed). Figure 6 shows the result, and the first-order finding is the one the argument rests on: running all three paradigms at once does not survive the removal of the symbols. SKELETONGRAPH’s first-search recall falls from 86% to 55% as the cues are withdrawn. Measured within task—the same issues under both conditions—that degradation is 0.125 on average, with a bootstrap 95% confidence interval of [0.010, 0.260] over the 26 paired tasks across both benchmarks. The built-in lexical baseline degrades as well, from 66% to 49%. Neither similarity over tokens, nor similarity in an embedding space, nor adjacency in a call graph withstands the loss of the symbol names, and running all three together does not change that.

We are deliberately more careful about the second-order question—whether SKELETONGRAPH’s *margin* over the baseline shrinks faster than the baseline itself does. The point estimates suggest it: the margin is +0.200 when the issue names the code and +0.065 when it does not, and on the decontaminated benchmark +0.160 against +0.056. But tested within task that difference is +0.058 with a 95% interval of [−0.093, 0.215], and only 5 of 26 tasks individually show it. At this sample

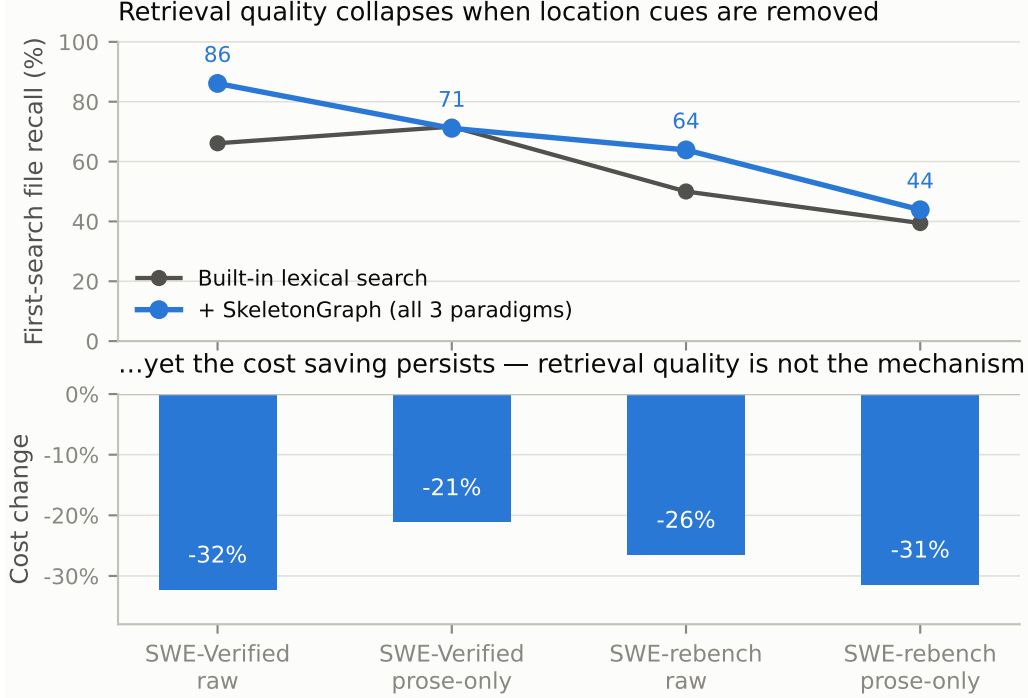


Figure 6: Top: first-search file recall as location cues are removed. **Bottom:** the cost saving, which persists throughout. SWE-Verified columns use the same 15 tasks as the prose run so the conditions are paired.

size we can establish that the retriever degrades sharply; we cannot establish that it degrades *to parity* with lexical search, and we do not claim it. The ceiling argument does not require that stronger statement—what matters is that combining every non-LLM paradigm available leaves symptom-only localization unsolved, which the first-order result shows on its own.

Table 6 gives the four cells numerically. Read the recall columns first: both systems lose accuracy as the cues are withdrawn, SKELETONGRAPH from 0.861 to 0.549 and the baseline from 0.661 to 0.493. The margin between them is smaller in the prose conditions than in the original ones, in the same direction on both benchmarks, though as noted above that second-order difference is not resolved at this sample size. The cost column behaves differently, and this is the comparison that carries the section: it stays between -21% and -31% in every cell, including the two where the retrieval margin has shrunk to a few points of recall. Whatever the exact size of the remaining retrieval advantage, it is not what the cost saving is tracking.

Four points of care belong with this table. The SWE-Verified rows are restricted to the same 15 tasks as the prose run so the raw and prose conditions are paired; that subset is *not* representative of the full 100, on which the saving is -14.6% rather than -32.2% , and the two must not be compared against each other. The effect of removing prose on cost is not monotone — on the memorized benchmark it shrinks the saving, on the unseen one it grows it — so we claim only that the saving persists, not that it trends. In the rebench prose-only cell one baseline task terminated in error and is excluded, leaving 14 baseline runs against 15. Finally, recall here uses the same rule as Table 1: tasks on which the agent never invoked SKELETONGRAPH are excluded from the recall average, because they record an adoption decision rather than a retrieval outcome. One, two and three tasks are excluded in the three prose and unseen cells respectively; cost and turn figures use every paired task, since adoption does not change what a run was billed. The convention matters more here than at $n = 100$ — scored the other way the prose-only cell reads -0.006 instead of $+0.065$ — which is

Benchmark	Issue text	First-search file recall		Change vs. built-in	
		Built-in $\rightarrow$ SKELETONGRAPH	Δ	Cost	Turns
SWE-Verified (memorized)	original	0.661 $\rightarrow$ 0.861	+0.200	−32.2%	−38.5%
	prose-only	0.696 $\rightarrow$ 0.762	+ 0.065	−21.1%	−22.6%
SWE-rebench (unseen)	original	0.577 $\rightarrow$ 0.737	+0.160	−26.4%	−35.6%
	prose-only	0.493 $\rightarrow$ 0.549	+ 0.056	−31.3%	−29.3%

Table 6: The decoupling, by condition. Retrieval accuracy degrades as location cues are withdrawn; the cost saving does not. All cells $n = 15$, paired, test-execution verified. The Δ column is a point estimate: at this sample size the difference between the original and prose-only margins is not statistically resolved (see text). Solve rate is a tie or ± 1 in every cell and is omitted.

itself a reason to treat these cells as directional and to prefer the larger sample for any quantitative claim.

Two conclusions follow, and they pull in opposite directions. The first is negative: no combination of non-LLM retrieval paradigms available to us performs the inference from symptom to responsible code. That inference is causal—it requires knowing what code *does* when it runs, not what it resembles—and similarity matching, whether lexical, dense, or topological, does not supply it. The second is positive, and is the more useful finding: **the cost saving does not track retrieval quality.** In the prose-only condition on the memorized benchmark, where SKELETONGRAPH retrieves barely better than the baseline, cost still falls by 21%. The decontaminated benchmark makes the same point more sharply: its largest cost saving (−31.3%) occurs in the cell with its *smallest* retrieval margin (+0.056), and its smallest saving in a cell with a larger one. Whatever SKELETONGRAPH is doing to reduce cost, it is not out-retrieving the baseline; it is bounding how far the agent wanders before it commits. This is the one claim in the section that does not depend on resolving the retrieval margin precisely—the cost effect is present and large in cells where that margin is, on any reading, small.

And iterating the retriever does not rescue it. A natural response is that a single search is the wrong unit—that an agent issuing follow-up queries would recover what the first missed. It does not, in the regime that matters. On the decontaminated set, additional structural searches return essentially the same ranking: 2.3 searches per task lift recall from its first-search value by +0.000 (Section 6 details this), because a query reformulated from the issue text alone re-samples the same impoverished vocabulary each time. A lexical agent’s own iteration fares better only because it has a genuine feedback loop—it reads a file, learns the repository’s real vocabulary, and greps again—which is precisely the behaviour a confident ranked list suppresses. Similarity retrieval cannot bootstrap information it does not already contain: if the responsible symbol is neither named in the query nor near it in the index, re-querying the same index will not surface it.

What the agent was actually missing was direction, not files. The two factors we crossed look independent but are the same quantity measured in two places. Memorization is knowledge of the repository held by the model; location cues are knowledge of the repository supplied by the issue text. Both answer the question *where should I look*, and ordering the four conditions by how much of that answer is available orders SKELETONGRAPH’s first-search recall exactly: 0.861 with both, 0.762 with memorization alone, 0.737 with cues alone, 0.549 with neither. Retrieval quality tracks the direction available to it, not the effort spent ranking. (The built-in baseline does not

order as cleanly—its prose-only value slightly exceeds its original-text value at $n = 15$ —so we rest this on the structural retriever’s ordering, which is monotone.)

The iteration data then shows where direction comes from when it is not given. The baseline acquires it *during the run*: it greps, reads a file, learns the repository’s actual vocabulary, and greps better, lifting recall by +0.128 to +0.268 over its first search. In both prose-only conditions this is decisive enough to overturn the ranking: the baseline’s cumulative recall finishes at 0.964 against SKELETONGRAPH’s 0.946 on the memorized benchmark, and at 0.715 against 0.632 on the unseen one. Given enough turns, the agent that explored the repository ended up locating the code *better* than the agent handed a ranked list. SKELETONGRAPH gets there in half the searches and for less money; it does not get further.

That inversion is the sharpest statement of what a retrieval layer is and is not. It delivers candidate files. It does not deliver knowledge of the repository, and a confident ranked list actively suppresses the process by which the agent would otherwise acquire that knowledge—exploration falls from 3.86 reads per task to 1.55. What the agent needs before it can edit is not a list of plausible files but sufficient confidence about which code is responsible and what changing it will do, and it will spend tokens until it has that regardless of how good the first result was. This is why recall and cost decouple: retrieval shortens the search for files, and the agent’s bill is set by the search for certainty.

What the wall is made of. The missing ingredient is not a sharper index but a different kind of knowledge. Locating the code responsible for a described symptom requires knowing what code *does* when it runs—its behaviour, and its effect on program state—which is exactly what none of the three paradigms encodes. Each matches on a form of resemblance: shared tokens (lexical), embedding proximity (dense), or graph adjacency (structural). None represents execution. This is why we attribute the ceiling to similarity-based localization rather than to our particular retriever, and it locates the open problem precisely: crossing it is not a retrieval task at all but a reasoning task over a representation of program behaviour, and building that representation—cheaply, without an LLM pass per repository—is where we see the next advance, not in a better ranking over the representations we already have.

We state the attribution this way because it is falsifiable, and we would rather name the test than assert the conclusion. A similarity-based retriever—no LLM in the loop, no execution trace—that recovers substantially more than lexical parity on symptom-only issues would refute it, and locate the ceiling in our implementation after all. What our evidence establishes is narrower than the general claim and, we think, sufficient to motivate the test: three paradigms run simultaneously, on two benchmarks, at two levels of memorization, do not clear it, and neither does iterating them.

7 Threats to Validity

Contamination. Our no-retrieval control establishes that SWE-bench Verified is memorization-saturated in our setting. We therefore treat solve rate as a tie-detection instrument, not a ranking signal. We additionally built a decontaminated replication path against a continuously-updated benchmark drawn from thousands of Python repositories with tasks postdating model cutoffs. On that decontaminated set (15 paired tasks, test-execution verified) the direction of every effect is preserved: cost −26.4%, turns −35.6%, solve rate 10/15 versus 9/15—statistically indistinguishable, as on the contaminated benchmark. The decontaminated replication therefore does not overturn any claim in this paper; it reproduces them on repositories the model has not memorized. Both of its conditions appear in Table 6.

Answer leakage through web tools. In one trace the agent used its web tools to fetch the upstream pull request corresponding to the benchmark task—that is, the reference patch. This is available to any web-enabled agent and is not specific to any retrieval arm, but it means solve rates for web-enabled frontier agents on public benchmarks should be treated with caution, and it is a further argument for decontaminated or private task sets.

Generality across agents. The deployment measurement—the tail decomposition of Table 3—is made against one production agent. Codex CLI, OpenHands, Gemini CLI and others differ in tool surface, default search policy, and system prompt, and we do not claim a specific effect size transfers to them. Two considerations bound how much this limits the result. First, the components of the argument are replicated outside that agent: the retrieval ordering holds in an agent-free ablation (Table 2), the cost and quality ordering holds in our own react loop on a different model family (Table 5), and the decoupling holds on a decontaminated benchmark (Table 6). The single-agent dependency is therefore confined to the magnitude of the deployment saving, not to the decoupling itself. Second, the most natural form of this objection—that our baseline agent has unusually capable built-in search, and a weaker agent would show a different picture—runs against the direction of the bias: a weaker native searcher would leave *more* exploration for retrieval to remove, so measuring against a strong baseline makes our estimate conservative. We note that [?], comparing retrieval strategies across three production agent CLIs, report substantial harness-dependent variation in outcomes, which is a reason to expect the *size* of our effect to move across agents even where its direction does not. The react-loop results are consistent with that, showing a larger retrieval effect against a weaker model. We would still regard replication on a second production agent as the most valuable single addition to this work.

Run-to-run variance. Agent trajectories are stochastic; single volatile tasks can move aggregate percentages by double digits at $n \approx 50$. We report distributions and paired tests rather than point estimates, and flag outliers explicitly. To distinguish the tail shape from sampling variation we also recomputed the decomposition at every checkpoint from $n = 20$ to $n = 100$: the median stays near zero and the 90th and 95th percentiles stay strongly negative throughout. The 75th percentile is the unstable column and we do not rest any claim on it.

Scope. Headline results are Python. SKELETONGRAPH parses ten languages, but we do not claim performance parity outside Python and do not report multi-language solve rates.

Cost and token accounting. Token counts are the full prompt the model receives on each turn—system prompt, tool definitions, accumulated conversation, and every tool result—not retrieval payloads in isolation. For the deployed agent this is the sum of fresh, cache-read, and cache-creation input, since the runtime prompt-caches the re-sent history; for the react loop it is the API’s reported prompt tokens. One consequence is worth stating because it runs against our own interest: SKELETONGRAPH registers seven tool definitions that occupy the re-sent prefix on every turn, and that overhead is charged to SKELETONGRAPH, so the comparison understates rather than flatters it. Cost within the deployment study is the runtime’s own reported billing, with its cache discounts applied; cost within the react loop is imputed from token counts under a fixed published price sheet applied identically to every arm. The two are never pooled, every reported change is paired within a single harness, and absolute dollars are indicative—the *ratios* are the meaningful quantity.

8 Discussion

8.1 What this means for building code-context systems

The design space of Section 2.2 can now be scored against measurement rather than intuition. Prompt injection is dominated: a map placed in the fixed prompt is re-sent every turn, which is why the repository-map system consumes an order of magnitude more tokens for no solve-rate benefit. Graph query and structural retrieval both pay per use and land within a factor of 1.5 of each other. Lexical search remains a genuinely strong baseline at file granularity. The differences among the pay-per-use designs are small relative to the difference between all of them and the injection design — so the first-order engineering decision is *when* context is delivered, not how cleverly it is selected.

For a builder, the actionable form of our result is: optimize the delivery schedule before optimizing the ranking. Ranking improvements act on ΔT through the localization prefix, which is already short. Delivery-schedule mistakes act on $\bar{c}$ multiplied by every remaining turn.

8.2 Why we are not claiming more

It would be straightforward to report this work as a 15% cost reduction with improved retrieval accuracy and leave the distribution out. We think that framing, which is standard in this category, is the reason the category’s claims do not replicate. A mean is not a promise a user experiences: half of our users’ tasks would see no saving at all. What they would see is that their worst tasks stop running away — which is a real benefit, precisely stated, and one we can defend per-task.

We also decline the solve-rate claim available to us. SKELETONGRAPH won 75 to 74. On a benchmark where a no-retrieval control scores 36–44%, that difference is not evidence of anything, and reporting it as a win would be the same error we criticize.

9 Conclusion

We set out to determine what better code retrieval buys an agent. The answer is narrower than the design intent and than the field’s claims, and it is structural rather than incidental.

Structural retrieval does what it was designed to do: it returns the function to edit, at a granularity lexical search does not possess, and it collapses exploratory navigation. But cost in an agent loop accumulates as $\sum_t c_t$, and retrieval moves only the localization prefix of T while leaving $\bar{c}$ unchanged — because the agent takes what it needs regardless of who hands it over, and everything taken early is re-sent for the rest of the trajectory. Push the payload below what the task requires and the agent fetches the remainder itself; push it above and the surplus is billed every turn. Both directions were tested, and both cost accuracy.

The consequence is a decoupling: an order-of-magnitude improvement in function-level retrieval yields no change in median cost and no change in solve rate, while removing 42.5% of cost at the 95th percentile. Retrieval is an anti-catastrophe layer. It bounds the worst case; it does not shift the typical one. Until an agent’s own turn count falls — which is a property of how it implements and validates, not of how it searches — context accumulates and cost grows with it.

We think this is the more useful claim for practitioners and the more honest one for the field, and we release the harness, frozen task lists, and per-task records so that it can be checked rather than trusted.

References

- [1] Paul Gauthier. Aider: AI pair programming in your terminal. <https://github.com/Aider-AI/aider>, 2024. Repository map built by PageRank over a tree-sitter symbol graph.
- [2] DeusData. Codebase-memory: A tree-sitter code knowledge graph as an MCP server. <https://github.com/DeusData/codebase-memory-mcp>, 2026.
- [3] Safi Shamsi. Graphify: Knowledge-graph RAG for codebases. <https://github.com/safishamsi/graphify>, 2024.
- [4] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents, 2024.
- [5] Joseph Townsend, Chandresh Pravin, Kwun Ho Ngan, and Matthieu Parizy. A study on the impact of fault localization granularity for repository-scale code repair tasks, 2026. Fujitsu Research of Europe.
- [6] Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. LocAgent: Graph-guided LLM agents for code localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, pages 8697–8727, Vienna, Austria, 2025.
- [7] Zora Zhiruo Wang, Akari Asai, Xinyan Velocity Yu, Frank F. Xu, Yiqing Xie, Graham Neubig, and Daniel Fried. CodeRAG-Bench: Can retrieval augment code generation? In *Findings of the Association for Computational Linguistics: NAACL*, 2025.
- [8] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [9] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, et al. OpenHands: An open platform for AI software developers as generalist agents. <https://github.com/All-Hands-AI/OpenHands>, 2024.
- [10] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023.
- [11] Yash Doke. Hiermem: Hierarchical memory for long-horizon chat agents. *Research Square*, 2026. URL <https://www.researchsquare.com/article/rs-10055780/v1>. Preprint, Version 1.
- [12] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [13] Nebius. SWE-rebench: A continuously updated, decontaminated benchmark of real-world software engineering tasks. <https://huggingface.co/datasets/nebius/SWE-rebench>, 2026. Continuously-updated leaderboard at <https://swe-rebench.com>.
- [14] Max Brunsfeld et al. tree-sitter: An incremental parsing system for programming tools. <https://tree-sitter.github.io>, 2018.

- [15] Stephen E. Robertson, Steve Walker, Susan Jones, Micheline M. Hancock-Beaulieu, and Mike Gatford. Okapi at TREC-3. *NIST Special Publication 500-226: Overview of the Third Text REtrieval Conference (TREC-3)*, pages 109–126, 1995.
- [16] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.
- [17] Anthropic. Model context protocol specification. <https://modelcontextprotocol.io>, 2024.